



EC-220 Computer System Architecture

Dr. Ahsan Shahzad

Department of Computer and Software Engineering (DC&SE)

College of Electrical and Mechanical Engineering (CEME)

National University of Science and Technology (NUST)

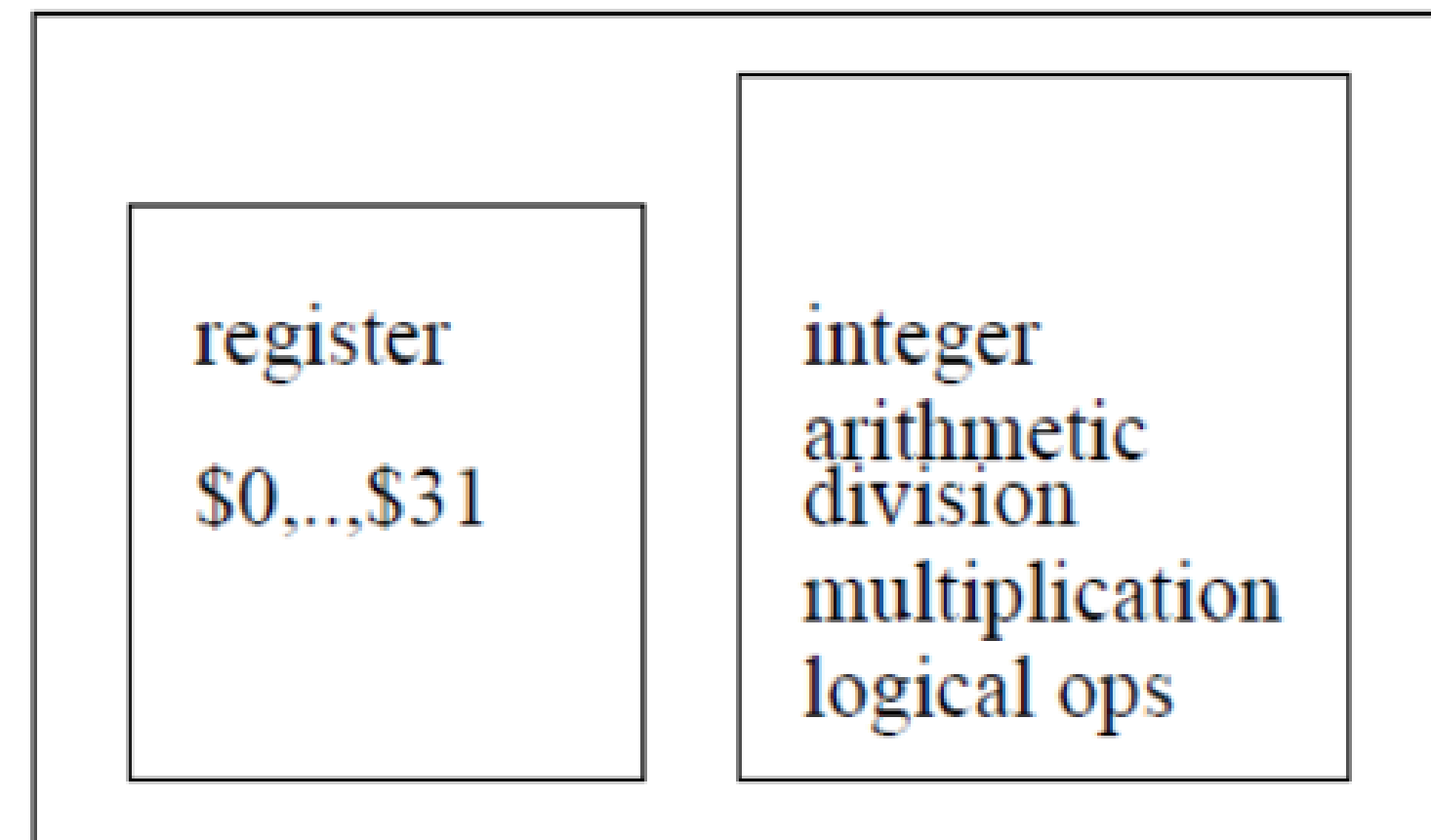
Week # 10_OLE

MIPS Co-Processors

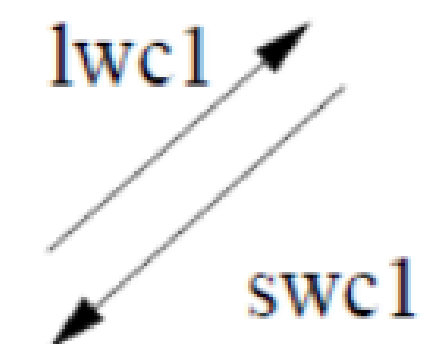
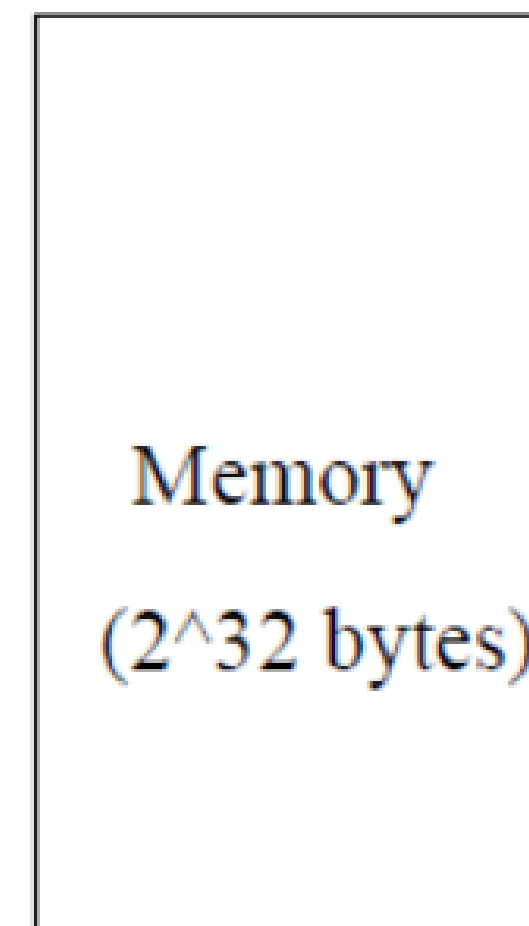
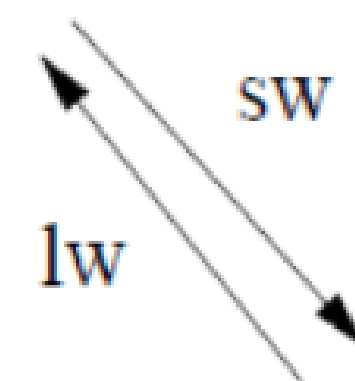
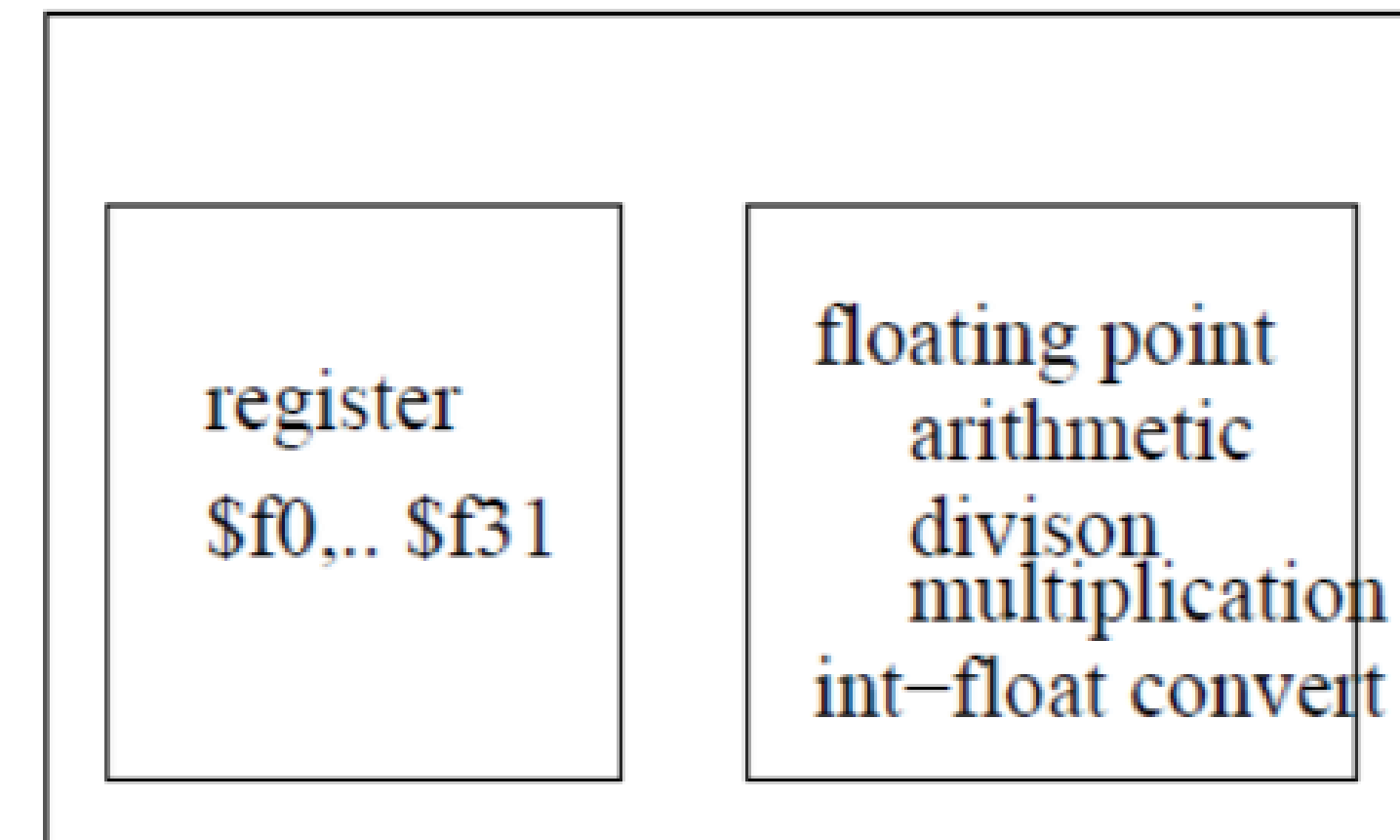
- **MIPS supports up to four coprocessors**
 - CP 0/1/2/3
- **C0 or CP0**
 - System control coprocessor or kernel processor
 - Essential part of main CPU or processor
 - Special Regs: EPC, Cause,....
- **C1 or CP1 or FPU**
 - (optional) Floating point processor or Floating point unit
 - Separate set of 32 registers (\$f0 - \$f31)
- **CP 2/3**
 - (optional) implementations-defined coprocessors
 - E.g. CP2 as Geometry transformation engine (3D computer graphics) in game consoles

MIPS Co-Processors

CPU (central processing unit)



FPU (floating point unit)
"coprocessor 1"



MIPS Floating Point Instructions

- **Separate set of instructions**
 - Used separate set of regs. \$f0 - \$f31
 - Executed by CP1 or FPU (one of the co-processors)
- **Floating point Registers (\$f0 - \$f31)**
 - Also called CP1 registers
 - Each reg. is 32 bits (for MIPS-32)
- **MIPS floating point instructions are also called CP1 instructions.**
 - Many CP1 instrs. have C1 in their name e.g. lwc1, swc1 etc.
 - CP1 Arithmetic instrs. Do not have c1 in their names, instead they include a */completer/*
 - Completers:
 - “.s ” Single-Precision Floating Point (32 bits)
 - “.d ” Double-Precision Floating Point (64 bits)
 - “.w ” Integer (32-bits)

MIPS Floating Point Instructions - **Operands**

▪ Double-Precision Operands

- 64 bit – 16 regs. -> “even-odd paired regs. \$f0-\$f31”
- Even Reg. no. will hold the left part (msb) and odd no. reg. will save the remaining right half
- Instrs. With “.d ” used pair of regs. And Operand register # must be even
- **Example:**
 - `add.d $f0, $f2, $f4 # { $f0,$f1 } = { $f2, $f3 } + { $f4, $f5 }`
 - `add.d $f0, $f2, $f5 # ILLEGAL in MIPS 32, because f5 is odd.`

▪ Immediate Operands and Constant Registers

- MIPS FP instrs. Do not take immediate values
 - `addi.s $f0, $f1, 2.3 # ILLEGAL, no immediate FP instructions.`
- There is no counterpart of integer reg. 0 (\$0 aka \$zero)
 - `add.s $f0, $f1, $f2 # f0 has the sum,`

MIPS Floating Point Instructions - **Types**

■ Arithmetic Operations

- E.g. add.s, add.d, mul.s, mul.d etc.
- **Example:**
 - Add double-precision (64-bit) operands
 - **MIPS Code:** add.d \$f0, \$f2, \$f4

■ Load and Store

- **Example:**
 - Load double (64-bit or 8-bytes in to two consecutive registers)
 - **MIPS Code:** ldc1 \$f0, 12(\$t0) # \$f0 = Mem[\$t0 + 12]
 - **Other Similar instrs.** lwc1, swc1, ldc1 (l.d), sdc1 (s.d)

MIPS Floating Point Instructions - **Types**

▪ **Move between Register Files**

- E.g. integer regs. to FP regs.
- **Example:**
 - **mtc1** (move to CP1) and **mfc1** (move from CP1)
 - MIPS Code: **mtc1 \$t0, \$f0** # \$f0 = \$t0 **Note that destination is “second” register here.**
 - MIPS Code: **mfc1 \$t0, \$f0** # \$t0 = \$f0

▪ **Data Type Conversion**

- Convert between floating-point and integer formats.
- Note: Values don't convert automatically, need to use these instrs.
- **To: s, d, w ; From s, d, w formats**
 - **General Format: cvt. TO. FROM fd, fs**
 - e.g. **cvt.d.w \$f0, \$f2** # \$f0 = convert_from_int_to_double(\$f2)

MIPS Floating Point Instructions - **Types**

■ Comparison Instructions

- FP comparison and condition code setting
- **Example:**
 - MIPS Code: `c.gt.d $f0, $f2` `#(cond = true, if $f0 > $f2)`

■ Conditional Branch

- Branch on floating point condition (True / False)
- **Example:**
 - MIPS Code: `bc1f Label` `# Branch CP1 [condition code] false`

MIPS Floating Point Instruction Encoding

- **From Book**

- Page 212-213

```
add.s    $s0, $s0, $s1 # NOT ALLOWED (add.s expects FPU registers)
add      $f0, $f2, $f2 # NOT ALLOWED (add expects CPU registers)
```

Floating Point Multiplication

- **From Book**
 - Page 206-211